

Métodos Especiais: toString(), equals() e hashCode()

Como imprimir e comparar objetos em Java

Linguagem de Programação II

Objetivos da aula

- Entender o que acontece quando imprimimos um objeto diretamente.
- Entender por que aparece uma saída parecida com `Aluno@5e91993f`.
- Criar o método `toString()` dentro de uma classe.
- Usar `toString()` automaticamente com `System.out.println(objeto)`.
- Entender por que `==` e `equals()` não comparam o conteúdo dos objetos por padrão.
- Sobrescrever `equals()` e `hashCode()` para comparar objetos pelo conteúdo.
- Reforçar o uso de **atributos privados** (encapsulamento) junto com esses três métodos.
- Praticar com exemplos: `Aluno`, `Produto`, `Endereco`, `Cliente`, `Pacote` e `Entregador`.

- Esta aula parte do pressuposto de que os alunos já conhecem:
 - classe e objeto;
 - construtor;
 - **encapsulamento** (atributos privados e métodos públicos);
 - métodos básicos (como *getters*);
 - criação de objetos no `main`.
- O foco agora é melhorar a forma como esses objetos, já encapsulados, aparecem na tela.

- **Atributos privados** (`private`) só podem ser acessados de dentro da própria classe.
- Para que o mundo de fora (como o `main`) consiga ler um valor, criamos um *getter*:
 - `public String getNome() { return nome; }`
- Em todos os exemplos de hoje, as classes terão atributos privados.
- A pergunta da aula é: **como imprimir esses objetos de forma legível, respeitando o encapsulamento?**

Problema: imprimir um objeto diretamente

```
public class Aluno {  
    private String nome;  
    private double nota;  
  
    public Aluno(String nome, double nota) {  
        this.nome = nome;  
        this.nota = nota;  
    }  
  
    public String getNome() { return nome; }  
    public double getNota() { return nota; }  
}
```

A classe está encapsulada (atributos privados, getters públicos), mas ainda não sabe se transformar em texto.

Usando no Main: sem toString()

```
public class Main {  
    public static void main(String[] args) {  
        Aluno aluno = new Aluno("Ana", 8.0);  
  
        System.out.println(aluno);  
    }  
}
```

Saída possível:

```
Aluno@5e91993f
```

Mesmo usando getName() e getNota(), o println sozinho ainda não mostra os dados.

O que significa Aluno@5e91993f?

- O Java tentou imprimir o objeto, mas não recebeu uma regra de impressão.
- Então ele mostrou uma representação padrão.
- Essa representação costuma ter este formato:
 - nome da classe;
 - símbolo @;
 - um código interno em hexadecimal.
- Para alunos iniciantes, podemos entender isso como: **o Java mostrou um código do objeto, e não os dados dele.**
- Chamar `getNome()` e `getNota()` um a um até resolveria o problema, mas seria repetitivo.

A solução: criar o toString()

- O método `toString()` ensina ao Java como transformar um objeto em texto.
- Ele deve retornar uma `String`.
- Quando usamos `System.out.println(objeto)`, o Java chama o `toString()` automaticamente.
- Assim, quem programa escolhe o formato da impressão, em um único lugar, dentro da própria classe.

Como escrever o toString()

Dentro da classe, escrevemos:

```
@Override
public String toString() {
    return "texto_que_representa_o_objeto";
}
```

- @Override: indica que estamos reescrevendo um método que já existe no Java.
- public String toString(): nome e retorno obrigatórios.
- return: devolve o texto que será exibido.

Atenção: toString() e atributos privados

- Os atributos continuam private: ninguém de fora da classe pode acessá-los diretamente.
- Porém, o toString() é um método **da própria classe**.
- Por isso, ele pode usar nome e nota diretamente, sem precisar chamar getNome() ou getNota().
- Já no main, que está **fora** da classe, o acesso continuaria sendo feito por getters, se necessário.
- Resumindo: **encapsulamento protege o acesso externo, mas não impede a própria classe de usar seus dados.**

Exemplo 1: Classe Aluno com toString()

```
public class Aluno {  
    private String nome;  
    private double nota;  
  
    public Aluno(String nome, double nota) {  
        this.nome = nome;  
        this.nota = nota;  
    }  
  
    public String getNome() { return nome; }  
    public double getNota() { return nota; }  
  
    @Override  
    public String toString() {  
        return "Aluno:_" + nome + "_|_Nota:_" + nota;  
    }  
}
```

Exemplo 1: usando Aluno no Main

```
public class Main {  
    public static void main(String[] args) {  
        Aluno aluno = new Aluno("Ana", 8.0);  
  
        System.out.println(aluno);  
    }  
}
```

Saída:

```
Aluno: Ana | Nota: 8.0
```

Chamada automática e chamada manual

As duas formas abaixo produzem a mesma saída:

```
Aluno aluno = new Aluno("Ana", 8.0);  
  
System.out.println(aluno);  
System.out.println(aluno.toString());
```

Saída:

```
Aluno: Ana | Nota: 8.0  
Aluno: Ana | Nota: 8.0
```

Normalmente usamos apenas `System.out.println(aluno)`. Note que `aluno.nome` continua proibido fora da classe; quem expõe o dado é o `toString()`.

Exemplo 2: Classe Produto

```
public class Produto {  
    private String nome;  
    private double preco;  
    private int quantidade;  
  
    public Produto(String nome, double preco, int quantidade) {  
        this.nome = nome;  
        this.preco = preco;  
        this.quantidade = quantidade;  
    }  
  
    public String getNome() { return nome; }  
    public double getPreco() { return preco; }  
    public int getQuantidade() { return quantidade; }  
  
    @Override  
    public String toString() {  
        return nome + " | Preço: R$" + preco +  
            " | Estoque: " + quantidade;  
    }  
}
```

Exemplo 2: usando Produto no Main

```
public class Main {  
    public static void main(String[] args) {  
        Produto p1 = new Produto("Mouse", 49.90, 10);  
        Produto p2 = new Produto("Teclado", 120.00, 5)  
            ;  
  
        System.out.println(p1);  
        System.out.println(p2);  
    }  
}
```

Saída:

```
Mouse | Preço: R$ 49.9 | Estoque: 10  
Teclado | Preço: R$ 120.0 | Estoque: 5
```

Exemplo 3: Classe Endereco sem toString()

```
public class Endereco {  
    private String rua;  
    private int numero;  
    private String cidade;  
  
    public Endereco(String rua, int numero, String  
        cidade) {  
        this.rua = rua;  
        this.numero = numero;  
        this.cidade = cidade;  
    }  
  
    public String getRua() { return rua; }  
    public int getNumero() { return numero; }  
    public String getCidade() { return cidade; }  
}
```

Mesmo encapsulada, se imprimirmos um objeto Endereco diretamente, a saída ainda será pouco útil.

Exemplo 3: imprimindo Endereco sem toString()

```
public class Main {  
    public static void main(String[] args) {  
        Endereco endereco = new Endereco("Rua das  
            Flores", 120, "Patos");  
  
        System.out.println(endereco);  
    }  
}
```

Saída possível:

```
Endereco@6d06d69c
```

O Java não mostrou rua, número nem cidade.

Exemplo 3: Classe Endereco com toString()

```
public class Endereco {  
    private String rua;  
    private int numero;  
    private String cidade;  
  
    public Endereco(String rua, int numero, String cidade) {  
        this.rua = rua;  
        this.numero = numero;  
        this.cidade = cidade;  
    }  
  
    public String getRua() { return rua; }  
    public int getNumero() { return numero; }  
    public String getCidade() { return cidade; }  
  
    @Override  
    public String toString() {  
        return rua + ", " + numero + " - " + cidade;  
    }  
}
```

Exemplo 3: imprimindo Endereco com toString()

```
public class Main {  
    public static void main(String[] args) {  
        Endereco endereco = new Endereco("Rua das  
            Flores", 120, "Patos");  
  
        System.out.println(endereco);  
    }  
}
```

Saída:

```
Rua das Flores, 120 - Patos
```

Exemplo 4: Classe Cliente com Endereco

```
public class Cliente {
    private String nome;
    private String telefone;
    private Endereco endereco;

    public Cliente(String nome, String telefone, Endereco endereco)
    {
        this.nome = nome;
        this.telefone = telefone;
        this.endereco = endereco;
    }

    public String getNome() { return nome; }
    public String getTelefone() { return telefone; }
    public Endereco getEndereco() { return endereco; }

    @Override
    public String toString() {
        return nome + " | Tel: " + telefone +
            " | Endereco: " + endereco;
    }
}
```

Quando concatenamos endereco, o Java também chama o toString()

Exemplo 4: usando Cliente no Main

```
public class Main {  
    public static void main(String[] args) {  
        Endereco end = new Endereco("Rua das Flores",  
            120, "Patos");  
        Cliente cliente = new Cliente("Maria", "  
            99999-1111", end);  
  
        System.out.println(cliente);  
    }  
}
```

Saída:

```
Maria | Tel: 99999-1111 | Endereco: Rua das Flores,  
120 - Patos
```

Atenção: se Endereco não tiver toString()

Suponha que Cliente tenha toString(), mas Endereco não tenha.

```
return nome + " | Endereco: " + endereco;
```

A saída poderia ficar assim:

```
Maria | Endereco: Endereco@6d06d69c
```

Por isso, quando uma classe encapsulada aparece como atributo de outra, também é importante criar o toString() dela – e não apenas getters.

Exemplo 5: Classe Pacote (atributos, construtor e getters)

```
public class Pacote {  
    private String descricao;  
    private double peso;  
    private Cliente cliente;  
    private Endereco destino;  
  
    public Pacote(String descricao, double peso,  
                  Cliente cliente, Endereco destino) {  
        this.descricao = descricao;  
        this.peso = peso;  
        this.cliente = cliente;  
        this.destino = destino;  
    }  
  
    public String getDescricao() { return descricao; }  
    public double getPeso() { return peso; }  
    public Cliente getCliente() { return cliente; }  
    public Endereco getDestino() { return destino; }  
}
```

Exemplo 5: Classe Pacote (toString())

```
public class Pacote {  
    // atributos, construtor e getters  
    // (slide anterior)  
  
    @Override  
    public String toString() {  
        return "Pacote:␣" + descricao +  
               "␣|␣Peso:␣" + peso + "␣kg" +  
               "\nCliente:␣" + cliente +  
               "\nDestino:␣" + destino;  
    }  
}
```

O `toString()` acessa `descricao`, `peso`, `cliente` e `destino` diretamente, mesmo sendo privados, pois está dentro da própria classe.

Exemplo 5: usando Pacote no Main

```
public class Main {  
    public static void main(String[] args) {  
        Endereco endereco = new Endereco("Rua das Flores", 120, "  
            Patos");  
        Cliente cliente = new Cliente("Maria", "99999-1111",  
            endereco);  
  
        Pacote pacote = new Pacote("Livro de Java", 1.2, cliente,  
            endereco);  
  
        System.out.println(pacote);  
    }  
}
```

Saída:

```
Pacote: Livro de Java | Peso: 1.2 kg  
Cliente: Maria | Tel: 99999-1111 | Endereco: Rua das Flores, 120 -  
    Patos  
Destino: Rua das Flores, 120 - Patos
```

Exemplo 6: vetor de objetos com toString()

```
public class Main {  
    public static void main(String[] args) {  
        Aluno[] alunos = new Aluno[3];  
  
        alunos[0] = new Aluno("Ana", 8.0);  
        alunos[1] = new Aluno("Bruno", 7.5);  
        alunos[2] = new Aluno("Carla", 9.0);  
  
        for (int i = 0; i < alunos.length; i++) {  
            System.out.println(alunos[i]);  
        }  
    }  
}
```

Exemplo 6: saída do vetor

```
Aluno: Ana | Nota: 8.0  
Aluno: Bruno | Nota: 7.5  
Aluno: Carla | Nota: 9.0
```

- O laço percorre o vetor.
- Cada posição guarda um objeto `Aluno`, com seus atributos privados.
- Ao imprimir `alunos[i]`, o Java chama o `toString()` do objeto naquela posição.

Exemplo 7: Classe Entregador (atributos, construtor e adicionarPacote)

```
public class Entregador {
    private String nome;
    private String veiculo;
    private Pacote[] pacotes = new Pacote[5];
    private int totalPacotes = 0;

    public Entregador(String nome, String veiculo) {
        this.nome = nome;
        this.veiculo = veiculo;
    }

    public void adicionarPacote(Pacote pacote) {
        if (totalPacotes < pacotes.length) {
            pacotes[totalPacotes] = pacote;
            totalPacotes++;
        }
    }
}
```

Os atributos pacotes e totalPacotes ficam protegidos; só o próprio Entregador pode alterá-los.

Exemplo 7: Classe Entregador (getters e toString())

```
public class Entregador {  
    // atributos, construtor e adicionarPacote  
    // (slide anterior)  
  
    public String getNome() { return nome; }  
    public String getVeiculo() { return veiculo; }  
    public int getTotalPacotes() { return totalPacotes  
        ; }  
  
    @Override  
    public String toString() {  
        return "Entregador:_" + nome +  
            "_|_Veiculo:_" + veiculo +  
            "_|_Total_de_pacotes:_" + totalPacotes;  
    }  
}
```

Exemplo 7: usando Entregador no Main

```
public class Main {  
    public static void main(String[] args) {  
        Endereco endereco = new Endereco("Rua_das_Flores", 120, "  
            Patos");  
        Cliente cliente = new Cliente("Maria", "99999-1111",  
            endereco);  
        Pacote pacote = new Pacote("Livro_de_Java", 1.2, cliente,  
            endereco);  
  
        Entregador entregador = new Entregador("Joao", "Moto");  
        entregador.adicionarPacote(pacote);  
  
        System.out.println(entregador);  
        System.out.println(pacote);  
    }  
}
```

- Sem `toString()`, o Java imprime uma representação padrão pouco útil.
- Com `toString()`, escolhemos como o objeto será exibido.
- `System.out.println(objeto)` chama `toString()` automaticamente.
- Atributos continuam `private`: o `toString()` consegue usá-los porque é um método da própria classe.
- Se uma classe usa outra classe encapsulada como atributo, as duas podem precisar de `toString()`.
- Isso melhora a leitura do console e facilita os testes no `main`, sem abrir mão do encapsulamento.

Problema:

- Crie uma classe Produto com atributos **privados**:
 - nome;
 - preco;
 - quantidade.
- Crie um construtor público para inicializar os atributos.
- Crie *getters* públicos para os três atributos.
- Crie o método toString() para exibir os dados do produto.
- No Main, crie três produtos e imprima todos na tela.

Gabarito: Classe Produto

```
public class Produto {
    private String nome;
    private double preco;
    private int quantidade;

    public Produto(String nome, double preco, int quantidade) {
        this.nome = nome;
        this.preco = preco;
        this.quantidade = quantidade;
    }

    public String getNome() { return nome; }
    public double getPreco() { return preco; }
    public int getQuantidade() { return quantidade; }

    @Override
    public String toString() {
        return nome + " | R$" + preco + " | Qtd:" + quantidade;
    }
}
```

Gabarito: Main com produtos

```
public class Main {  
    public static void main(String[] args) {  
        Produto p1 = new Produto("Caderno", 25.0, 10);  
        Produto p2 = new Produto("Caneta", 2.5, 50);  
        Produto p3 = new Produto("Mochila", 120.0, 3);  
  
        System.out.println(p1);  
        System.out.println(p2);  
        System.out.println(p3);  
    }  
}
```

Problema:

- Crie as classes `Endereco`, `Cliente` e `Pacote`, todas com atributos **privados** e *getters*.
- O `Cliente` deve ter nome, telefone e endereço.
- O `Pacote` deve ter descrição, peso, cliente e destino.
- Crie `toString()` nas três classes.
- No `Main`, crie um endereço, um cliente e um pacote.
- Imprima o pacote no console como uma etiqueta de entrega.

```
public class Endereco {  
    private String rua;  
    private int numero;  
    private String cidade;  
  
    public Endereco(String rua, int numero, String cidade) {  
        this.rua = rua;  
        this.numero = numero;  
        this.cidade = cidade;  
    }  
  
    public String getRua() { return rua; }  
    public int getNumero() { return numero; }  
    public String getCidade() { return cidade; }  
  
    @Override  
    public String toString() {  
        return rua + ", " + numero + " - " + cidade;  
    }  
}
```

Gabarito: Classe Cliente

```
public class Cliente {
    private String nome;
    private String telefone;
    private Endereco endereco;

    public Cliente(String nome, String telefone, Endereco endereco)
    {
        this.nome = nome;
        this.telefone = telefone;
        this.endereco = endereco;
    }

    public String getNome() { return nome; }
    public String getTelefone() { return telefone; }
    public Endereco getEndereco() { return endereco; }

    @Override
    public String toString() {
        return nome + " | Tel: " + telefone +
            " | Endereco: " + endereco;
    }
}
```

Gabarito: Classe Pacote (atributos, construtor e getters)

```
public class Pacote {  
    private String descricao;  
    private double peso;  
    private Cliente cliente;  
    private Endereco destino;  
  
    public Pacote(String descricao, double peso,  
                  Cliente cliente, Endereco destino) {  
        this.descricao = descricao;  
        this.peso = peso;  
        this.cliente = cliente;  
        this.destino = destino;  
    }  
  
    public String getDescricao() { return descricao; }  
    public double getPeso() { return peso; }  
    public Cliente getCliente() { return cliente; }  
    public Endereco getDestino() { return destino; }  
}
```

Gabarito: Classe Pacote (toString() – etiqueta)

```
public class Pacote {  
    // atributos, construtor e getters  
    // (slide anterior)  
  
    @Override  
    public String toString() {  
        return "---_ETIQUETA_" +  
            "\nPacote:_" + descricao +  
            "\nPeso:_" + peso + "_kg" +  
            "\nCliente:_" + cliente +  
            "\nDestino:_" + destino;  
    }  
}
```

Gabarito: Main da etiqueta

```
public class Main {  
    public static void main(String[] args) {  
        Endereco endereco = new Endereco("Rua_das_Flores", 120, "  
            Patos");  
  
        Cliente cliente = new Cliente("Maria", "99999-1111",  
            endereco);  
  
        Pacote pacote = new Pacote("Livro_de_Java", 1.2, cliente,  
            endereco);  
  
        System.out.println(pacote);  
    }  
}
```


Saída esperada da etiqueta

```
--- ETIQUETA ---  
Pacote: Livro de Java  
Peso: 1.2 kg  
Cliente: Maria | Tel: 99999-1111 | Endereco: Rua das Flores, 120 -  
        Patos  
Destino: Rua das Flores, 120 - Patos
```

Mais um problema: comparando objetos

- Já sabemos **exibir** um objeto de forma clara com `toString()`.
- Mas como saber se **dois objetos são iguais?**
- Exemplo: dois objetos `Aluno` com o mesmo nome e a mesma nota representam o mesmo aluno?
- Para responder isso, vamos conhecer mais dois métodos especiais: `equals()` e `hashCode()`.

Problema: comparando com ==

```
public class Main {  
    public static void main(String[] args) {  
        Aluno a1 = new Aluno("Ana", 8.0);  
        Aluno a2 = new Aluno("Ana", 8.0);  
  
        System.out.println(a1 == a2);  
    }  
}
```

Saída:

false

O operador == compara **referências** (endereços de memória), não o conteúdo. a1 e a2 são objetos diferentes, mesmo com os mesmos dados.

O equals() padrão tem o mesmo problema

```
System.out.println(a1.equals(a2));
```

Saída:

```
false
```

- Toda classe Java herda um equals() da classe Object.
- Por padrão, esse equals() também compara apenas a referência – o mesmo que ==.
- Para comparar pelo **conteúdo**, precisamos sobrescrever o equals().

A solução: sobrescrever equals()

- O método `equals()` decide quando dois objetos devem ser considerados iguais.
- Ele segue um contrato:
 - **reflexivo**: `a.equals(a)` é sempre `true`;
 - **simétrico**: `a.equals(b)` é igual a `b.equals(a)`;
 - **transitivo**: se `a.equals(b)` e `b.equals(c)`, então `a.equals(c)`;
 - **consistente**: chamadas repetidas devem dar sempre o mesmo resultado;
 - `a.equals(null)` deve ser sempre `false`.

Como escrever o equals()

```
import java.util.Objects;

@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null || getClass() != obj.getClass()) return false;

    Aluno outro = (Aluno) obj;
    return Double.compare(nota, outro.nota) == 0 &&
        Objects.equals(nome, outro.nome);
}
```

- `this == obj`: se for o mesmo objeto, já retorna true.
- `getClass() != obj.getClass()`: garante que é da mesma classe.
- `Objects.equals(...)`: compara atributos com segurança, mesmo se forem null.

Exemplo: Aluno com equals()

```
import java.util.Objects;

public class Aluno {
    private String nome;
    private double nota;

    public Aluno(String nome, double nota) {
        this.nome = nome;
        this.nota = nota;
    }

    public String getNome() { return nome; }
    public double getNota() { return nota; }

    @Override
    public String toString() {
        return "Aluno:␣" + nome + "␣|␣Nota:␣" + nota;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return
            false;
    }
}
```

Exemplo: usando equals() no Main

```
public class Main {  
    public static void main(String[] args) {  
        Aluno a1 = new Aluno("Ana", 8.0);  
        Aluno a2 = new Aluno("Ana", 8.0);  
  
        System.out.println(a1 == a2);  
        System.out.println(a1.equals(a2));  
    }  
}
```

Saída:

```
false  
true
```


Por que sobrescrever hashCode() também?

- Existe outro contrato em Java: **se `a.equals(b)` é true, então `a.hashCode()` deve ser igual a `b.hashCode()`.**
- Estruturas como `HashSet` e `HashMap` usam o `hashCode()` para organizar os objetos internamente.
- Se sobrescrevermos `equals()` e esquecermos o `hashCode()`, essas estruturas podem se comportar de forma inesperada (por exemplo, aceitar objetos "duplicados").
- Regra prática: **sempre sobrescreva os dois juntos.**

Como escrever o hashCode()

```
import java.util.Objects;

@Override
public int hashCode() {
    return Objects.hash(nome, nota);
}
```

- `Objects.hash(...)` combina os atributos em um único código numérico.
- Use **os mesmos atributos** que aparecem no `equals()`.

Exemplo: Aluno completo (equals() + hashCode())

```
public class Aluno {  
    // atributos, construtor, getters e toString()  
    // (slides anteriores)  
  
    @Override  
    public boolean equals(Object obj) {  
        if (this == obj) return true;  
        if (obj == null || getClass() != obj.getClass()  
            ()) return false;  
        Aluno outro = (Aluno) obj;  
        return Double.compare(nota, outro.nota) == 0  
            &&  
            Objects.equals(nome, outro.nome);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(nome, nota);  
    }  
}
```

equals() e hashCode() em coleções

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        HashSet<Aluno> turma = new HashSet<>();

        turma.add(new Aluno("Ana", 8.0));
        turma.add(new Aluno("Ana", 8.0));

        System.out.println(turma.size());
    }
}
```

Saída:

1

Sem equals()/hashCode() sobrescritos, o HashSet trataria os dois objetos como diferentes, e o tamanho seria 2.

Resumo: equals() e hashCode()

- == compara referência; equals() sobrescrito compara conteúdo.
- equals() e hashCode() devem ser sobrescritos **juntos**.
- Objects.equals() e Objects.hash() ajudam a escrever essas comparações com segurança.
- Atributos privados continuam acessíveis dentro de equals() e hashCode(), pelo mesmo motivo do toString(): são métodos da própria classe.
- HashSet, HashMap e outras coleções dependem desse contrato para funcionar corretamente.

Problema:

- Use a classe Produto (já encapsulada) criada anteriormente.
- Adicione um `equals()` que compare nome, preço e quantidade.
- Adicione o `hashCode()` correspondente.
- No Main, crie dois objetos Produto com os mesmos dados.
- Compare-os com `==` e com `equals()`, e imprima os resultados.

Gabarito: Produto (equals() e hashCode())

```
public class Produto {  
    // atributos, construtor, getters e toString()  
    // (slides anteriores)  
  
    @Override  
    public boolean equals(Object obj) {  
        if (this == obj) return true;  
        if (obj == null || getClass() != obj.getClass()) return  
            false;  
        Produto outro = (Produto) obj;  
        return Double.compare(preco, outro.preco) == 0 &&  
            quantidade == outro.quantidade &&  
            Objects.equals(nome, outro.nome);  
    }  
  
    @Override  
    public int hashCode() {  
        return Objects.hash(nome, preco, quantidade);  
    }  
}
```

Gabarito: Main comparando produtos

```
public class Main {  
    public static void main(String[] args) {  
        Produto p1 = new Produto("Mouse", 49.90, 10);  
        Produto p2 = new Produto("Mouse", 49.90, 10);  
  
        System.out.println(p1 == p2);  
        System.out.println(p1.equals(p2));  
    }  
}
```

Saída:

```
false  
true
```